

Informe de la Conexión y API Externa en el Simulador Bancario

Resumen de la Conexión y la API Externa

El proyecto **Simulador Bancario** implementa una integración simulada con una API externa para gestionar transferencias. Esto incluye endpoints para **iniciar transferencias salientes** (envío de pagos) y para **confirmar o recibir transferencias entrantes** (con verificación OTP). La arquitectura actual utiliza JWT para autenticar peticiones externas, genera códigos OTP para la confirmación de operaciones y notifica resultados por medio de una URL configurada. Sin embargo, la revisión revela varios **problemas de seguridad y consistencia** en este diseño. En particular, se encuentran **validaciones insuficientes**, **duplicación de lógica OTP**, **falta de controles de permiso** y **manejo inconsistente del proceso de confirmación**. A continuación se detalla el funcionamiento actual y se señalan los errores identificados, seguidos de recomendaciones paso a paso para mejorarlos.

Proceso de Transferencia Saliente (Endpoint `/api/transferencia/`)

Flujo actual: Cuando un usuario (por ejemplo, un oficial bancario) inicia una transferencia saliente mediante una petición POST a `/api/transferencia/`, el sistema espera recibir un **JWT de autenticación** en la cabecera. El endpoint (`api_send_transfer`) verifica el token JWT y luego intenta parsear el cuerpo JSON de la solicitud ¹. Los datos esperados siguen (en teoría) el esquema SEPA, incluyendo campos como `payment_id`, `debtor_account_id`, importe, IBANs, etc. Si no se proporciona un `payment_id`, el código toma un identificador idempotente de la cabecera `Idempotency-Id` para evitar duplicados ².

Tras autenticar y recopilar datos, la vista llama al servicio de transferencias: `TransferService.ingest_transfer(data)` ³. Esta función realiza varias tareas importantes:

- **Idempotencia:** Si ya existe una transferencia con el mismo `payment_id`, simplemente la retorna para no duplicar ⁴.
- **Rate limiting:** Si se han iniciado demasiadas transferencias recientemente desde la misma cuenta deudora (más de 5 en 5 minutos), el servicio crea directamente la transferencia con estado **RJCT (rechazada)** para simular un rechazo por límite excedido ⁵.
- **Creación de la transferencia:** Si pasa las validaciones anteriores, registra la nueva transferencia en la base de datos con estado **PDNG (Pendiente)** ⁶ y programa una tarea asíncrona (Celery) para procesarla en 5 minutos ⁷. En este punto también se genera un código OTP de 6 dígitos asociado a la operación, creando un registro `OTPChallenge` con estado "CREATED" ⁸.

Una vez `TransferService.ingest_transfer` devuelve la transferencia (pendiente), la vista `api_send_transfer` **vuelve a generar un código OTP** adicional por su cuenta y lo guarda como otro

`OTPChallenge` en la base de datos ⁹. Finalmente, responde al cliente con un JSON que incluye el `payment_id` de la transferencia, su estado (`PDNG`) y un `challenge_id` para identificar el reto OTP, indicando que se requiere OTP (`"otp_required": true`) ¹⁰. **Nota:** En esta respuesta **no se envía el código OTP al cliente**, solo el identificador del desafío. Se asume que el OTP sería entregado al usuario final por otro canal (SMS, correo, etc.), aunque en este entorno simulado no se implementó envío real.

Problemas observados en transferencias salientes:

- **Falta de validación de datos:** El endpoint no valida que el JSON entrante tenga todos los campos necesarios ni su formato. Por ejemplo, si falta `debtor_account_id` u otro campo obligatorio, la función `TransferService.ingest_transfer` lanzará excepciones o errores (p.ej., un `KeyError` al intentar acceder a `data["debtor_account_id"]` ¹¹). No se usan serializadores ni formularios para validar la estructura SEPA, lo que deja espacio a **datos incompletos o inválidos** sin manejo adecuado. Esto fue señalado como un riesgo: *"Endpoint /api/transfer_incoming permite datos arbitrarios y solo valida existencia de JWT, sin controlar estructura completa"* ¹² (aplicable igualmente a `api_send_transfer`).
- **Duplicación de generación de OTP:** Existe una doble creación de códigos OTP para la misma transferencia. El servicio de dominio ya genera y guarda un OTP (como parte de `ingest_transfer` en `TransferService`) ⁸, y luego la vista vuelve a crear otro `OTPChallenge` independiente ⁹. Esto podría provocar confusión o inconsistencias acerca de cuál OTP es válido. Es un indicio de **lógica duplicada** y diseño desacoplado entre capa de servicio y vista.
- **Ausencia de controles de autorización granular:** Si bien el endpoint requiere un JWT válido, actualmente **no verifica que la cuenta deudora utilizada pertenezca o esté autorizada para el usuario autenticado**. Cualquier usuario con un JWT válido podría, en teoría, iniciar una transferencia desde cualquier cuenta si conoce/indica los IDs, pues no hay verificación de pertenencia. Esto representa un problema de **seguridad**: debería comprobarse que el usuario (por ejemplo, un oficial bancario o cliente) solo pueda operar sobre sus cuentas o las que deba manejar.
- **Exención de CSRF sin precauciones adicionales:** La vista está marcada con `@csrf_exempt` (heredado porque esta ruta API no usa la protección CSRF tradicional) ¹³. En aplicaciones JSON API con JWT esto puede ser aceptable, pero se debe asegurar que sólo agentes legítimos llamen al endpoint. Dado que se usa JWT en Authorization header, el riesgo CSRF se mitiga; aun así, conviene mencionar que **todas las rutas sensibles deberían protegerse adecuadamente**, ya sea manteniendo CSRF (si el front-end comparte dominio) o mediante tokens seguros en encabezados.
- **Respuesta 202 (Accepted) con OTP pendiente:** El endpoint devuelve código HTTP 202 indicando aceptación pendiente. Esto es correcto para representar que la transferencia fue recibida y está a la espera de confirmación OTP. Sin embargo, el hecho de no proveer el OTP en la respuesta dificulta la prueba manual (en el ambiente de simulación, **otros endpoints** proporcionan el OTP directamente para facilitar pruebas, como veremos). En un entorno real, lógicamente el OTP no se devuelve en la API sino que se enviaría al usuario por un canal externo; aquí simplemente notamos la diferencia de comportamiento entre endpoints simulados.

Proceso de Transferencia Entrante y Confirmación OTP

Hay **dos endpoints** involucrados en completar la transferencia pendiente mediante OTP en este proyecto:

1. `/api/transferencia/verify/` – pensado para que el cliente aplique el código OTP y confirme la transferencia.
2. `/api/transferencias/entrantes/` (o `/payments`) – aparentemente para que *el sistema externo* notifique la entrada/ejecución de una transferencia, incluyendo la verificación OTP. Este diseño es confuso, ya que mezcla la lógica de desafío OTP con la recepción de notificaciones. Básicamente, se usa también para confirmar la transferencia, pero responde de forma distinta.

Veamos cada uno:

- **Confirmación OTP por el usuario** (`api_verify_otp`): Este endpoint espera una petición POST con JWT en el header (el mismo token obtenido al iniciar sesión via `/api/login/`) y un cuerpo JSON con el `payment_id` de la transferencia y el `otp` ingresado por el usuario ¹⁴. Al recibirlos, busca en la base de datos un registro `OTPChallenge` que corresponda al `payment_id` y OTP proporcionados, con estado "CREATED" (no usado aún) ¹⁵. Si no lo encuentra, responde con error OTP inválido. Si el OTP es correcto, marca el challenge como "USED" (usado) ¹⁶.

Luego intenta recuperar la transferencia pendiente por `payment_id`. **Aquí hay una sección problemática:** si la transferencia **no existe**, el código crea objetos "dummy" en la base de datos para Debit, Creditor, cuentas, etc., únicamente para poder crear un objeto `Transfer` mínimo y así finalizar el flujo ¹⁷ ¹⁸. Esta inserción forzada sugiere que en algunas condiciones la transferencia podría no haberse creado en el paso anterior – por ejemplo, si `TransferService.ingest_transfer` falló por validación y lanzó excepción antes de crearla. En lugar de abortar, el código intenta **fabricar datos ficticios** para completar el proceso, lo cual es una práctica riesgosa e inconsistente (se terminan creando entidades "Dummy" en la BD ¹⁷ sin control). Esto claramente fue implementado para evitar romper la secuencia si faltó la `Transfer`, pero no es una solución correcta en producción.

Finalmente, con la transferencia existente (o recién creada ficticiamente), `api_verify_otp` procede a **marcarla como aceptada** (`status = 'ACCP'`) y asigna el identificador de quien autorizó (campo `auth_id` con el usuario del JWT) ¹⁹. Responde al cliente con el estado final ("ACCP") y el `payment_id`. En este flujo, la transferencia se da por confirmada inmediatamente tras verificar OTP, *sin esperar a la tarea asíncrona*. **Importante:** nótese que **no se realiza ninguna verificación de saldo ni descuento al balance de la cuenta en este endpoint**, simplemente se cambia el estado a aceptada. Esto contrasta con el flujo asíncrono programado (que sí actualiza saldos) y puede llevar a incoherencias financieras.

- **Recepción de transferencia entrante** (`api_transfer_incoming`): Este endpoint, también sin CSRF, parece simular la notificación desde el lado externo (por ejemplo, la entidad bancaria que confirma la transferencia tras OTP). Sin embargo, su comportamiento es peculiar. Al hacer POST a `/api/transferencias/entrantes/`, se exige o un JWT válido o una sesión autenticada tradicional ²⁰. Luego se intenta leer el JSON entrante ²¹. Este JSON puede venir en dos formatos:
- **Primer paso (sin OTP):** Si **no incluye** un campo `otp` (es decir, es la primera vez que el sistema externo "golpea" este endpoint), el código espera al menos un `payment_id` ²². Si falta `payment_id`, error. Si existe, el sistema genera un OTP de 6 dígitos (`otp_val`) y crea un objeto

`OTPChallenge` con ese OTP y el `payment_id` dado ²³. Luego responde inmediatamente con `challenge_id` y `otp_required: true`, **incluyendo el OTP en claro en la respuesta** (`'otp': otp_val`) ²⁴. Esta respuesta simula que el sistema externo solicita un segundo factor y **proporciona el código OTP** (esto está hecho para facilitar las pruebas dentro del simulador – en un caso real, la entidad externa jamás devolvería el OTP al cliente final en la misma petición).

- **Segundo paso (con OTP):** Si la petición entrante sí trae un `otp` (y además un código TOTP temporal `totp` para 2FA adicional) ²⁵, entonces el flujo intenta validar ese OTP. Primero comprueba el TOTP mediante `verify_totp` (un método no mostrado aquí, posiblemente verifica un código temporal tipo Google Authenticator) ²⁶. Si el TOTP no es válido, se rechaza la petición. Luego verifica que exista un `OTPChallenge` con el `payment_id` y OTP dados y estado "CREATED" ²⁷. Si no existe, responde OTP inválido. Si todo cuadra, marca el `OTPChallenge` como "USED" ²⁸.

En este punto, curiosamente, el endpoint **llama de nuevo** a `TransferService.ingest_transfer(data)` con el mismo payload original ²⁹. Dado que en el primer paso posiblemente ya se había creado la transferencia (o al menos registrado un `OTPChallenge`), esta segunda llamada normalmente encontrará la transferencia existente por `payment_id` y la retornará sin cambios (recordemos que `ingest_transfer` devuelve la existente si ya estaba registrada) ⁴. Solo si no existía la creará en ese momento. Finalmente responde con el `payment_id` y el `status` actual de la transferencia (que seguramente seguirá siendo 'PDNG' en ese momento, ya que `ingest_transfer` no cambia el estado salvo a RJCT en casos de límite, o deja PDNG) ²⁹. Nótese que **este endpoint no marca la transferencia como aceptada**; se limita a crearla o recuperarla después de OTP. Pareciera que la intención es que la confirmación final ocurra luego (quizá vía la tarea asíncrona programada a los 5 minutos, que verá el OTP usado y procederá).

Problemas observados en confirmación/entrantes:

- **Flujos redundantes y confusos:** Existe una dualidad entre usar `api_transfer_incoming` vs `api_verify_otp` para confirmar la transferencia. Ambos reciben OTP y ambos tocan la base de datos, pero cada uno hace cosas ligeramente distintas. Esta duplicación puede llevar a incoherencias. Por ejemplo, `api_verify_otp` marca la transferencia como ACCP de inmediato ¹⁹, mientras `api_transfer_incoming` no lo hace, dejando la transferencia en PDNG a la espera de la tarea Celery. Dos caminos distintos para un mismo fin complican el mantenimiento y pueden provocar que, dependiendo de cuál se use, el resultado sea distinto (e.g., uno descuenta saldo y otro no, uno notifica externo y otro no, etc.). Lo ideal sería **unificar el proceso de confirmación** en un solo flujo bien definido.
- **OTP entregado en respuesta (seguridad):** En el flujo `transfer_incoming`, el OTP se devuelve en la respuesta JSON ²⁴. Si bien esto se hizo para facilitar la simulación (por ejemplo, en pruebas automatizadas el cliente puede obtener el OTP directamente), es una práctica insegura en un entorno real. Divulgar el OTP por la misma vía de solicitud compromete el segundo factor. En producción, los OTP deberían entregarse por canales fuera de banda (o al menos no retornarse a la misma entidad que lo solicitó). Dado que el proyecto es un simulador, podemos entender esta decisión, pero conviene resaltarla.
- **Creación de datos dummy:** Como mencionado, `api_verify_otp` crea entidades ficticias si no encuentra la transferencia ¹⁷. Esto indica un diseño deficiente, pues en lugar de asegurar la existencia previa de la transferencia antes de solicitar OTP, se intenta parchear a posteriori. Tal

situación no debería ocurrir: idealmente, si `api_send_transfer` falló al crear la transferencia, no se debería ni generar OTP. Este bloque de código “*crear Dummy Debtor/Creditor*” es señal de un error lógico previo y puede insertar datos basura en el sistema.

- **Falta de conciliación con tarea asíncrona:** Cuando la confirmación sucede vía `api_verify_otp`, el estado pasa a ACCP inmediatamente, pero **no se realiza la deducción del saldo ni se notifica a la API externa** en ese momento. Se confía en que la tarea programada (`process_transfer_task`) correrá luego de 5 minutos. Sin embargo, dado que la tarea comprueba el estado de la transferencia al iniciar y **no hace nada si la transferencia ya no está en PDNG** ³⁰, en este caso la tarea abortará sin ejecutar la lógica de descuento ni notificación. Esto significa que si un usuario confirma la OTP rápidamente vía `api_verify_otp`, **la transferencia quedará aceptada sin descontar fondos** (el balance no cambia) y **no se enviará notificación externa** (porque la tarea sale temprano) – a menos que manualmente se maneje ese caso. Esta inconsistencia es grave, pues rompe la integridad financiera simulada. En el flujo alternativo (no usando `api_verify_otp`, dejando que la tarea procese tras 5 minutos), sí se descuentan fondos y se notifica al endpoint externo configurado ³¹ ³². La coexistencia de ambos enfoques no está bien coordinada.
- **Autorización laxa en entrantes:** Similar al caso saliente, `api_transfer_incoming` solo comprueba que exista *alguna* autenticación (JWT o usuario logueado) ²⁰, pero no valida quién llama ni si tiene derecho a confirmar esa operación. En un entorno real, la notificación de confirmación vendría de la entidad bancaria externa, posiblemente autenticada de forma distinta (por ejemplo con un token de sistema). Aquí cualquier usuario autenticado podría enviar un OTP para cualquier `payment_id`. No hay verificación de que el usuario de la sesión JWT sea el mismo que inició la transferencia o tenga permiso para confirmarla. Este es otro problema de seguridad, aunque en la simulación quizás el JWT representaba al mismo oficial siempre.
- **Manejo de errores mínimo:** En ambos endpoints de confirmación, el manejo de errores se limita a devolver JSON de error y códigos HTTP apropiados en caso de OTP/TOTP incorrecto o datos faltantes, lo cual está bien. Pero no hay logging de estos eventos (fallos de OTP, etc.), que podría ser útil para auditoría. Tampoco hay diferenciación de errores (por ejemplo, OTP expirado vs OTP incorrecto – todos son “OTP inválido”).

Notificación a la API Externa y Conexión

Como parte del flujo completo, una vez la transferencia se procesa (sea vía tarea asíncrona o confirmación manual), el sistema **notifica el resultado a un endpoint externo** configurado. Esto se realiza en la tarea Celery `process_transfer_task` al final de su ejecución: tras marcar la transferencia como aceptada (ACCP) o rechazada (RJCT), construye un payload JSON con `payment_id`, estado final, IBAN de la cuenta deudora y monto ³², y hace un HTTP POST a la URL definida en `settings.SIMULATOR_NOTIFY_URL` ³³.

Esta URL (por defecto `http://localhost/notify` según `.env.example`) representaría un servicio externo que recibe la notificación de transferencia completada. Cabe destacar que el código envía esta petición sin mucha robustez: está envuelta en un try/except genérico que **silencia cualquier excepción** de `requests` (timeout, conexión rechazada, etc.) ³⁴. Solo se deja un comentario indicando que se podría reintentar o loguear el fallo, pero actualmente simplemente pasa de largo en caso de error. Por tanto, **si la notificación no llega al externo, el sistema no avisa ni reintenta**, lo cual podría dejar transferencias sin confirmar externamente.

Adicionalmente, la configuración de URLs externas en `settings.py` muestra que los endpoints del simulador están parametrizados (`SIMULADOR_API_URL`, `SIMULADOR_TOKEN_URL`, etc.) ³⁵. Esto sugiere que el sistema podría también actuar como cliente de otra API (por ejemplo, para obtener un token OAuth externo o enviar transferencias hacia fuera). Sin embargo, en el código proporcionado no se observa una implementación de cliente saliente consumiendo `BASE_URL` o rutas externas – más bien el simulador mismo **expone** rutas equivalentes (`/oidc/token`, `/otp/single`, `/payments`) para emular cómo un tercero accedería a él. En resumen, la “conexión externa” en este contexto está limitada a la notificación de resultados y a la simulación de endpoints de autenticación/OTP que un tercero utilizaría.

Problemas observados en la conexión externa:

- **Notificaciones no fiables:** Como se indicó, la notificación por POST al externo carece de manejo de reintentos o registro de fallos. En sistemas reales, habría que asegurar la entrega (por ejemplo, reintentando varias veces exponencialmente, o al menos dejando constancia para un operador).
- **Seguridad de la comunicación:** No se aprecia en el código el uso de HTTPS u otras medidas; la URL de notificación por defecto es HTTP. Para producción, esto debería ser HTTPS y posiblemente con autenticación (ej: firma de mensajes, token compartido, etc.). Dado que es un simulador local, se entiende el uso de localhost sin TLS, pero es un punto a mencionar para robustez.
- **Endpoint `/payments` público:** El endpoint de entrada de notificaciones (`/api/transferencias/entrantes/`) está expuesto y actualmente acepta llamadas desde cualquier fuente con un JWT válido. De nuevo, en un esquema real, la entidad externa debería autenticarse de manera específica (por ejemplo, con un token de sistema diferente al de usuario). Dejar este endpoint abierto a cualquier portador de JWT es un riesgo si ese JWT pudiera ser de un usuario común.
- **Configuración en código:** Aunque gran parte de las URLs externas se configuran vía `.env` (lo cual es bueno), se encontró en la revisión general que hay **rutas absolutas y credenciales sensibles** incluidas en algunos archivos de configuración (por ejemplo, archivos de despliegue en `docs/` o configuraciones de Nginx/Tor). Esto no afecta directamente al código de transferencia, pero forma parte de los hallazgos en la auditoría general del proyecto ³⁶ ³⁷.

Errores y Riesgos Identificados

Resumiendo los hallazgos, listamos los principales **errores y riesgos** en el manejo de la conexión API externa y transacciones entrantes/salientes:

- **Validaciones Insuficientes de Entrada:** No se verifica la integridad del JSON recibido en los endpoints de transferencia. Esto puede provocar excepciones (ej. falta de campos) o permitir datos inválidos que luego causen comportamientos inesperados ¹¹.
- **Doble Creación de OTP:** Existe duplicación de lógica al generar OTPChallenges en `ingest_transfer` y de nuevo en la vista `api_send_transfer` ⁹ ⁸. Esto es innecesario y puede causar inconsistencias (por ejemplo, OTP diferentes registrados para un mismo `payment_id`).
- **Proceso de Confirmación Inconsistente:** Dos endpoints distintos (`transfer_incoming` vs `verify_otp`) manejan la confirmación OTP de maneras diferentes, llevando a resultados distintos en estado de transferencia y saldo. Esto genera **condiciones de carrera** entre la confirmación manual y la tarea programada, pudiendo dejar transferencias aceptadas sin descuento de fondos.
- **Lógica de “parche” con datos dummy:** La creación de objetos Dummy en `api_verify_otp` ¹⁷ evidencia un manejo incorrecto de errores previos. Este código podría insertar datos falsos en BD y enmascara el verdadero problema (transferencia no creada correctamente en el paso inicial).

- **Falta de Control de Permisos Granular:** Cualquier usuario autenticado (JWT válido) puede intentar enviar o confirmar transferencias, sin comprobar relación con las cuentas afectadas ni rol específico. No hay verificación de rol en los endpoints API (a diferencia de vistas web donde usan `@user_passes_test` para limitar ciertas acciones). Esto abre vectores de abuso si no se gestiona cuidadosamente quién obtiene JWT.
- **Endpoints CSRF Exentos:** Varias rutas API están exentas de CSRF ³⁸, lo cual es comprensible para APIs RESTful, pero siempre debe considerarse el contexto. Si en algún momento estas API fueran consumidas desde un front-end en el mismo dominio, podría ser necesario habilitar CSRF o utilizar cabeceras especiales. Es un punto a vigilar.
- **Notificaciones Externas Silenciadas:** La falta de logging o reintento en caso de fallo al notificar al sistema externo puede provocar que no se detecten problemas de conectividad. En un entorno real, esto afectaría la confiabilidad de todo el flujo interbancario.
- **Ausencia de Pruebas Automáticas:** No se encontraron tests específicos para estos flujos (de hecho no hay carpeta `tests/` en el proyecto). Esto significa que errores como los anteriores pueden pasar desapercibidos hasta una prueba manual. La auditoría ya había destacado la falta de tests unitarios/integración ³⁹.

Recomendaciones de Mejora Paso a Paso

Para abordar los problemas anteriores, a continuación se presentan recomendaciones **paso a paso** enfocadas en mejorar la vista de envío de transferencias y el proceso de confirmación entrante. Estas acciones buscan aplicar buenas prácticas de desarrollo Django y seguridad, incrementando la robustez de la “conexión” con la API externa simulada:

1. Validar exhaustivamente los datos de entrada en las vistas API de transferencias:

Implementar validaciones sobre el JSON recibido antes de procesarlo. Se puede lograr de varias formas: utilizando **forms de Django** adaptados a JSON, incorporando **serializers de Django REST Framework**, o manualmente comprobando cada campo requerido. Por ejemplo, asegúrate de que en `api_send_transfer` lleguen campos esenciales como `debtor_account_id`, `creditor_account_id`, montos numéricos válidos, IBAN con formato correcto, etc. Si falta algo o un formato es erróneo, devolver un error 400 claro (“*Faltan datos*” o “*Formato inválido*”). En el código actual solo se valida la presencia de `payment_id` y, en `api_transfer_incoming`, del OTP/TOTP, pero no se valida la estructura SEPA completa. Añadir estas validaciones evitará llamadas a `TransferService.ingest_transfer` con datos mal formados que puedan causar excepciones no controladas ¹¹.

```
- *Implementación:* Podrías crear un **serializer o esquema** SEPA (campos de
deudor, acreedor, cuentas, importe, etc.) y usarlo en ambos endpoints. Si se
detecta un error, responder con
`JsonResponse({'error': 'detalle del problema'}, status=400)` antes de cualquier
acción en BD. Esto mejora la seguridad (no operaciones con datos inválidos) y la
**usabilidad** de la API (respuestas de error más descriptivas).
```

2. Eliminar la doble generación de OTP y centralizar la lógica de desafío:

Decide dónde debe generarse y almacenarse el código OTP para la transferencia y **hazlo solo una vez**. Hay dos opciones razonables: - **a)** Generar OTP únicamente en la capa de servicio

(`TransferService.ingest_transfer`) y modificar `api_send_transfer` para **no** volver a generarlo. En lugar de crear un segundo `OTPChallenge`, la vista podría reutilizar el challenge creado en `ingest_transfer`. Actualmente `ingest_transfer` ya crea un `OTPChallenge` con `payment_id` y `otp` ⁸, aunque no devuelve directamente ese código. Se podría ajustar para que `ingest_transfer` retorne también el OTP generado (o el objeto `OTPChallenge`), de forma que la vista pueda, por ejemplo, enviarlo o usar su ID. - **b)** Alternativamente, mover la generación de OTP por completo a la vista (`api_send_transfer`) y evitar que `TransferService` lo haga. Esto implicaría quitar o desactivar la parte de OTP dentro de `ingest_transfer` ⁸. La vista crearía el `OTPChallenge` una vez obtenida la transferencia PDNG. Esta opción quizás simplifica el flujo (la capa de servicio solo se encarga de la transferencia y la tarea, y la vista maneja OTP). No obstante, habría que tener cuidado de también programar la tarea Celery en la vista en este caso, ya que actualmente está en `ingest_transfer`.

Cualquiera de las dos vías elimina la duplicación. La opción (a) es atractiva porque deja la lógica de negocio (persistencia de OTP) en un solo lugar transaccional. Optando por (a), la implementación sería: **usar el OTP ya generado**. Por ejemplo, tras `transfer = TransferService.ingest_transfer(data)`, buscar el `OTPChallenge` asociado a ese `payment_id` (que acabamos de crear) para obtener el `otp`. Incluso se podría incluir ese OTP en la respuesta para pruebas (solo en entorno dev), o al menos loguearlo para que el tester lo vea. Así evitamos la inconsistencia de tener 2 OTP posibles.

Nota: Si decides mantener la creación dual por algún motivo (no recomendado), al menos asegura que ambos OTP sean sincronizados (p.ej., usar el mismo código en ambos lugares). Pero la mejor práctica es uno solo.

3. Unificar el mecanismo de confirmación de OTP (flujos entrantes):

Simplifica la forma en que una transferencia pendiente se confirma con OTP, en lugar de tener dos endpoints haciendo cosas similares. Aquí se recomienda: - **Utilizar un solo endpoint de confirmación OTP**. Probablemente `api_verify_otp` sea el candidato, ya que está pensado para recibir `payment_id` y OTP directamente del cliente que autentica. Podrías **eliminar o deshabilitar** el uso de `api_transfer_incoming` para la confirmación de OTP interactiva. En su lugar, el flujo sería: cliente inicia transferencia -> recibe notificación de OTP requerido -> cliente llama a `/api/transferencia/verify/` con el OTP. El endpoint `api_transfer_incoming` entonces podría reservarse para *otras finalidades* o pruebas automatizadas, pero no para el usuario final. - **Asegurar consistencia en el procesamiento al confirmar OTP**: Cuando llegue la confirmación OTP (por el endpoint unificado), se debe **completar la transferencia de forma correcta**. Esto implica: - Validar nuevamente autenticación (JWT) y la información (`payment_id`, `otp`). - Marcar el `OTPChallenge` como usado/confirmado (esto ya se hace). - Finalizar la transferencia **descontando saldo y cambiando estado a exitoso**. Actualmente `api_verify_otp` marca ACCP pero no descuenta dinero ¹⁹, mientras la tarea Celery descuenta pero podría no correr. Deberíamos integrar esas acciones. Por ejemplo, al confirmar OTP, invocar la misma lógica de negocio que hace la tarea Celery: verificar fondos y actualizar balance y estado. Esto se puede refactorizar extrayendo esa parte de `process_transfer_task` ³¹ ⁴⁰ a una función reutilizable (ej. un método en `TransferService` tipo `complete_transfer(transfer)` que realice las operaciones 1), 2), 3) de la tarea). De esta manera, `api_verify_otp` tras validar OTP podría llamar a `TransferService.complete_transfer(transfer)` para aplicar los cambios inmediatos. - En ese escenario, **habría que cancelar o marcar la tarea Celery** programada para que no procese de nuevo esa transferencia. Dado que la tarea comprueba `if transfer.status != 'PDNG': return` ³⁰, no causaría daño (simplemente terminaría al ver ACCP). Pero dejaría de notificar al externo. Por ello, tras completar inmediatamente la transferencia en `api_verify_otp`, **también notifica al servicio externo**

en ese momento. Se puede reutilizar la URL `SIMULATOR_NOTIFY_URL` y hacer un POST igual que hace la tarea ³², pero ahora mismo en la respuesta a OTP. Así, el sistema externo se entera al instante. Para evitar duplicaciones, tal vez convenga anular la tarea programada (aunque al ser 5 min, el cliente externo ya estaría notificado, y cuando la tarea corra simplemente saldrá sin hacer nada relevante salvo quizá intentar notificar de nuevo; eso podría provocar un doble intento de notificación). Lo óptimo sería descolarla si es posible (por ejemplo, cancelar la tarea en la cola Celery si la librería lo permite, o marcar en la Transfer algún flag). - **Eliminar la creación de objetos dummy:** Si unificas bien el flujo, no debería darse el caso de `payment_id` inexistente al verificar OTP. Toda transferencia que requiera OTP debe haberse creado sí o sí al iniciar el proceso. Por tanto, el bloque de código de creación de Dummy Debtor/Creditor puede eliminarse. En su lugar, si `Transfer.objects.get(payment_id=..)` no encuentra nada, eso indicaría un error lógico y se debe retornar error (p.ej., "Transferencia no encontrada" 404) en vez de crear datos en blanco. Idealmente nunca pasará porque se garantizó creación en el paso inicial.

En resumen, se busca que la **confirmación OTP sea atómica y completa**: verificación + finalización + notificación, evitando caminos paralelos.

4. Fortalecer la autorización y permisos en estos endpoints:

Actualmente se confía en que cualquier usuario con JWT pueda usar los endpoints. Deberías introducir **filtros de permiso** más estrictos: - En `api_send_transfer`, si el modelo de usuarios lo permite, asocia la transferencia al usuario que la inició (quizás vía `auth_id` o mediante el `OficialBancario` si corresponde). Luego verifica, por ejemplo, que el usuario autenticado esté autorizado a mover fondos de la cuenta deudora indicada. Dado que en el simulador un `OficialBancario` no parece tener relación directa con un Debtor (deudor) específico, quizás habría que modelarlo o al menos asumir que el oficial puede operar ciertas cuentas. En entornos reales, se implementaría que solo el dueño o un oficial habilitado pueda iniciar transferencias desde una cuenta dada. - En `api_transfer_incoming` o el endpoint unificado de confirmación, considerar usar un **método de autenticación distinto para la entidad externa**. Por ejemplo, podría requerir una API key especial o un JWT distinto al de usuarios normales. Así te aseguras que solo el sistema externo legítimo confirme transferencias (esto importa más en producción; en el simulador puede omitirse si se simplifica a que el mismo usuario confirma). - Usar decoradores Django de permiso cuando sea posible. Por ejemplo, si decides exponer ciertos endpoints solo a superusuarios u oficiales, podrías aplicar `@user_passes_test` o verificar roles. En la configuración actual, muchos endpoints web usan `user.is_superuser` para restringir cambios administrativos ⁴¹; para la API JWT podrías incluir en el payload del token info de rol y chequearla manualmente.

Estas mejoras cierran posibles brechas de seguridad y hacen el comportamiento más predecible (nadie que no deba puede invocar la transferencia o confirmarla).

5. Revisitar la necesidad de `api_transfer_incoming` y su funcionalidad:

Luego de los pasos anteriores, evalúa si `api_transfer_incoming` sigue siendo necesario. Si decidiste unificar todo en `api_send_transfer` + `api_verify_otp`, este endpoint quizá quede obsoleto. Podrías: - **Opción A:** Eliminarlo para evitar confusiones (asegurando que ningún cliente lo use en tests). - **Opción B:** Re-aprovecharlo para *otro propósito simulado*, por ejemplo, simular *transferencias entrantes reales ajenas al banco* (dinero que llega a las cuentas desde fuera). El nombre sugiere que quizás se pensó para algo así. Pero su implementación actual está muy ligada al OTP, por lo que Opción A suele ser más sensata a corto plazo. - Si por motivos de compatibilidad con alguna prueba lo mantienes, al menos refactorízalo para que actúe de acuerdo con los cambios: podría simplemente invocar internamente a la misma lógica de `api_verify_otp` o de finalización. Pero evitar mantener dos caminos distintos.

6. Manejar la notificación externa de forma más robusta:

Mejorar el manejo de la **conexión HTTP externa** al notificar resultados: - Añade al menos un **log de error** cuando `requests.post` falle al llamar a `SIMULATOR_NOTIFY_URL` ³⁴. Actualmente el except está silencioso. Podrías usar Python logging para registrar que la notificación de cierta transferencia no se pudo enviar, incluyendo quizás el `payment_id` y motivo de la excepción. Esto ayudaría en depuración. - Si es factible, implementar **reintentos** automáticos. Por ejemplo, usando Celery puedes re-encolar otra tarea de notificación si falla, con algún retraso. O simplemente intentar X veces dentro de `process_transfer_task` antes de rendirse. - Validar la respuesta del servidor externo: actualmente se ignora. En un caso real, uno querría quizás comprobar código 200 OK del receptor, para confirmar que recibió la notificación. Si no, igualmente reintentar. - Configurar `SIMULATOR_NOTIFY_URL` para entornos de despliegue (posiblemente una URL de un sistema de pruebas). Asegurarse que en producción no quede apuntando a localhost. - Opcionalmente, **securizar la notificación**: enviar junto con el payload algún token o firma para que el receptor pueda autenticar que proviene del simulador legítimo. Dado que es un banco simulador notificando a otro sistema, suele haber acuerdos de seguridad (IP fijas, tokens, etc.). Esto ya es un detalle más de arquitectura externa, pero se menciona por completitud.

7. Registrar eventos y eliminar datos ficticios:

Aprovechar el modelo `LogTransferencia` definido en la aplicación para **registrar eventos clave** del flujo (inicio de transferencia, generación de OTP, confirmación OTP, errores, etc.). Por ejemplo, al generar un OTP se podría crear un log tipo 'OTP' con contenido "OTP generado para payment_id X". Al confirmar, otro log 'TRANSFER' con "Transfer X confirmada por usuario Y". Esto brindará trazabilidad. Actualmente no vimos código usando activamente `LogTransferencia`, pero está definido con tipos de log interesantes ⁴².

Asimismo, eliminar los "Dummy data" creados durante las pruebas de OTP (si ya no se usarán). Si en la base de datos quedaron Debtor "Dummy Debtor" o Creditor "Dummy Creditor" creados por pruebas anteriores ¹⁷, conviene limpiarlos para no contaminar los datos reales. Tras aplicar las mejoras, esos ya no se crearán más.

8. Añadir pruebas automatizadas del flujo de transferencia:

Como paso final, se recomienda encarecidamente **escribir tests** que cubran estos casos: - Caso exitoso de transferencia: iniciar transferencia, obtener OTP, confirmar OTP, verificar que el estado final sea ACCP, el saldo se haya descontado correctamente y la notificación externa (simulada) se haya "enviado" (puedes simular el receptor externo con un endpoint de prueba que registre la recepción). - Caso de fondos insuficientes: simular que `DebtorAccount.balance < monto` y comprobar que la transferencia termina en RJCT ³¹. - Caso de OTP incorrecto o expirado: verificar que no se confirma la transferencia si se manda un OTP erróneo (debe seguir en PDNG o finalmente ser cancelada tras timeout). - Casos de límite de velocidad: enviar más de 5 transferencias en menos de 5 minutos y confirmar que la 6ª es creada directamente como RJCT ⁵. - Caso de fallo en notificación: esto es difícil de probar sin un mecanismo de callback; podrías configurar `SIMULATOR_NOTIFY_URL` a algo inválido y ver que al menos se loguea el error.

Estos tests evitarán regresiones en el futuro y afianzarán que la "conexión externa" funciona como se espera en diferentes escenarios.

Aplicando estos pasos, el sistema deberá quedar más sólido y seguro. En síntesis, las mejoras se centran en **validar lo que entra, unificar la lógica de OTP y confirmación, garantizar que el dinero se descuenta y**

registre correctamente, y reforzar la comunicación externa. Con esto, la simulación de las transacciones entrantes y salientes se acercará más a un entorno real controlado, con menos casos inesperados y mayor claridad en el flujo. Cada recomendación apuntada corrige uno o más de los hallazgos del análisis, llevando el módulo de transferencias a un estado más confiable y mantenible.

Finalmente, no olvidemos documentar estos cambios en el README o la documentación de la API simulada, para que los desarrolladores futuros entiendan cómo deben usarse los endpoints (especialmente si cambian o se eliminan algunos como resultado de esta refactorización). Una buena documentación y ejemplos actualizados complementarán las mejoras de código en la experiencia global del proyecto. ⁴³

1 2 3 9 10 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 38 41 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/banco/views.py

4 5 6 7 8 11 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/services/transfer_services.py

12 36 37 39 43 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/docs/prompt_informe

13 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/banco/urls.py

30 31 32 33 34 40 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/banco/tasks.py

35 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/simulador_banco/settings.py

42 **GitHub**

https://github.com/markmur90/Simulador/blob/f81b1e98b78e379b733cd3fa70351f2f5b4e4729/simulador_banco/banco/models.py